



人工智能专业综合实训指导书

题目：中英文语境下图像隐喻识别和情感分类评测

From:2026CACC 评测-赛道七-任务一

专业：人工智能

编写版本：2026 版

选题来源：From:CCAC2026：中国情感计算大会

<https://mp.weixin.qq.com/s/Q3PyGX67RGVaCLDtWZuV9g>

1.实训目的

本次实训旨在掌握多模态技术在图像隐喻识别与情感分类中的应用原理与实现流程，理解视觉符号、文本语义和隐喻修辞之间的内在关联，熟练运用图像文本编码器、思维链、检索增强生成、LoRA、GRPO 等相关技术；围绕职场女性性别歧视主题，完成中英文双语境下图像隐喻识别、情感倾向与情感强度分类的联合任务，提升对多模态数据中隐含语义与复杂社会态度的解析能力；基于深度学习、提示工程、大语言模型微调三类主流技术路线设计创新方案，通过中英文数据集联合任务平均准确率开展实验评测，对比分析不同方法的性能、优势与不足，同时探究语言语境差异对任务效果的影响，最终锻炼算法设计、实验分析能力，推动多模态感知与情感计算技术在社会舆情分析等场景下的落地应用。

2.实训内容

本实训内容来自第六届中国情感计算大会（CCAC 2026）技术评测，任务分为 3 个部分：图像隐喻识别、图像情感分类以及文档撰写。

任务细节以及数据集获取详见：

<https://github.com/nlp-sentiment/Metaphor-recognition-and-sentiment>

2.1 方法列举

（1）基于深度学习的方法

本方法针对图像隐喻识别与情感分类任务搭建深度学习框架：首先采用 ViT 等视觉编码器提取图像的基础特征；其次结合大语言模型，运用思维链（CoT）知识蒸馏技术，将多模态大模型挖掘的图像描述等隐含信息转化为辅助特征；最后依托注意力机制实现特征融合，完成隐喻识别与情感分类任务。基于深度学习的实现方案，核心环节是搭建模型并进行模型训练。

（2）基于提示工程的方法

本方法针对图像隐喻识别与情感分类任务搭建提示工程框架：结合思维链提示、检索增强生成与自我校验机制搭建分层结构化提示体系，逐层解析图像内容中的隐喻逻辑与语义关联，完成隐喻识别与情感分类任务。基于提示工程的实现方案，核心环节是构建适配任务的提示指令。

（3）基于大语言模型微调的方法

本方法面向图像隐喻识别与情感分类任务搭建大语言模型微调框架：首先调用参数量更大的教师模型生成高质量标注样本、思维链推理过程及任务示范数据等信息；再利用这些生成信息基于 LoRA 轻量化微调、GRPO 强化学习算法对开源大语言模型（学生模型）开展参数

迭代与策略优化；最终依托训练完成的大语言模型完成隐喻识别与情感分类任务。基于大语言模型微调的实现方案，核心环节是制定微调策略，并对预训练大模型执行参数更新与微调。

2.2 中英文语境下的图像隐喻识别任务（1-2 天完成）

2.2.1 任务描述

该任务要求识别图像中是否存在隐喻表达，并解析其语义映射关系。重点考察模型在职场歧视主题下，对视觉符号（喻体）与其指向的本体之间的逻辑捕捉能力。

输入：原始图像（中/英文场景）。

输出：判定是否存在隐喻。

2.2.2 任务要求

（1）选取 2.1 中的两种不同的方法设计具体的模型，分别在中文、英文数据集上开展实验，总计完成四组对照实验，实现中英文语境下的图像隐喻识别任务。隐喻识别任务选用准确率与 F1 值作为评价指标。

（2）对模型进行训练和验证，比较各个模型的表现，并通过调参和改进模型架构提高识别精度，将最终的结果通过图或者表格展示。

2.2.3 方法参考

（1）基于深度学习的方法参考：

首先利用 ViT 图像编码器提取原图的基础视觉特征；随后调用视觉大语言模型 API，输入原图并搭配两组提示词：“描述这张图像中可直观看到的视觉内容”与“结合图像细节，说明其想要表达的隐藏含义”，生成对应的两段文本。接着采用 RoBERTa 文本编码器分别提取两段文本的特征，拼接后得到融合文本辅助特征。最后通过交叉注意力机制完成视觉基础特征与文本辅助特征的交互融合，并将融合后的特征向量送入分类层，完成隐喻识别任务。

（2）模型实现细节：

图像基础特征提取

采用 ViT 作为图像编码器，对输入原始图像进行视觉特征提取。

参考代码如下：

```
class ImageFeatureExtractor(nn.Module):
    """基于 ViT 的图像特征提取器"""

    def __init__(self, model_name="google/vit-base-patch16-224", feature_dim=768):
        super().__init__()
        self.vit = ViTModel.from_pretrained(model_name)
        self.feature_dim = feature_dim
        # 添加一个投影层将 ViT 输出映射到固定维度
        self.projection = nn.Linear(768, feature_dim)

    def forward(self, pixel_values):
```

```
# 提取 ViT 特征
outputs = self.vit(pixel_values)

# 使用[CLS] token 的输出作为全局特征
cls_features = outputs.last_hidden_state[:, 0, :] # [batch_size, 768]

# 投影到目标维度
visual_features = self.projection(cls_features)

return visual_features
```

基于视觉大语言模型的文本生成

为补充图像语义信息，通过接口调用方式使用开源视觉大语言模型 API。将同一张原始图像作为输入，依次设置两组定向提示词：描述这张图像中可直观看到的视觉内容、结合图像细节，说明其想要表达的隐藏含义，分别生成视觉描述文本与深层语义解读文本两段内容，从表层视觉、深层内涵两个维度丰富语义表达。

参考代码如下：

```
class VLLMTextGenerator:
    """基于视觉大语言模型的文本生成器"""

    def __init__(self, api_key=None, base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
                  model="qwen3.7-plus"):
        self.client = OpenAI(
            api_key=api_key or os.getenv("DASHSCOPE_API_KEY"),
            base_url=base_url,
        )
        self.model = model
        self.visual_prompt = "描述这张图像中可直观看到的视觉内容。"
        self.hidden_prompt = "结合图像细节，分析并说明其想要表达的隐藏含义。"

    def _image_to_url(self, image_path):
        """将图像路径转换为URL 格式"""
        if os.path.exists(image_path):
            import base64
            with open(image_path, "rb") as f:
                return f"data:image/jpeg;base64,{base64.b64encode(f.read()).decode('utf-8')}"
        return image_path

    def _generate(self, image_url, prompt):
        """生成单段文本"""
        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=[{
                    "role": "user",
                    "content": [
                        {"type": "image_url", "image_url": {"url": image_url}},
                        {"type": "text", "text": prompt},
                    ],
                }],
            )
            return response.choices[0].message.content
        except Exception as e:
            print(f"生成失败: {e}")
            return None
```

```

        max_tokens=500,
        temperature=0.1,
    )
    return response.choices[0].message.content
except Exception as e:
    print(f"生成文本出错: {e}")
    return ""

def generate_texts(self, image_path):
    """生成视觉描述和隐藏含义两段文本"""
    image_url = self._image_to_url(image_path)
    visual_text = self._generate(image_url, self.visual_prompt)
    hidden_text = self._generate(image_url, self.hidden_prompt)
    return visual_text, hidden_text

```

文本辅助特征提取与拼接

使用预训练 RoBERTa 文本编码器对上述两段生成文本分别进行编码。首先对文本执行分词、编码、位置嵌入等预处理操作，输入 RoBERTa 模型后提取每段文本对应的全局语义特征。将两组文本特征在特征维度上进行拼接融合，得到整合了视觉描述与语义解读的文本辅助特征。

参考代码如下：

```

class TextFeatureExtractor(nn.Module):
    """基于 RoBERTa 的文本特征提取器"""

    def __init__(self, model_name="roberta-base", feature_dim=768):
        super().__init__()
        self.roberta = RobertaModel.from_pretrained(model_name)
        self.tokenizer = RobertaTokenizer.from_pretrained(model_name)
        self.feature_dim = feature_dim
        # 投影层
        self.projection = nn.Linear(768, feature_dim)

    def forward(self, texts):
        # 处理输入
        if isinstance(texts, str):
            texts = [texts]

        # 分词和编码
        encodings = self.tokenizer(
            texts,
            padding=True,
            truncation=True,
            max_length=128,
            return_tensors="pt"
        )

```

```

# 移动到相同设备
device = next(self.roberta.parameters()).device
encodings = {k: v.to(device) for k, v in encodings.items()}

# 提取特征
outputs = self.roberta(**encodings)
# 使用[CLS] token 的输出
cls_features = outputs.last_hidden_state[:, 0, :] # [batch_size, 768]
# 投影
text_features = self.projection(cls_features)

return text_features

```

多模态特征交互与融合

引入交叉注意力机制搭建跨模态交互模块，以 ViT 输出的图像基础特征、RoBERTa 得到的文本辅助特征作为双路输入。通过交叉注意力计算图像特征与文本特征之间的关联权重，完成两种模态特征的深度交互、信息互补，输出融合图像与文本语义的统一多模态特征向量。

参考代码如下：

```

class CrossModalFusion(nn.Module):
    """ 基于交叉注意力的多模态特征融合模块 """

    def __init__(self, feature_dim=768, num_heads=8, dropout=0.1):
        super().__init__()
        self.feature_dim = feature_dim

        # 图像到文本的交叉注意力
        self.image_to_text_attention = nn.MultiheadAttention(
            feature_dim, num_heads, dropout=dropout, batch_first=True
        )
        # 文本到图像的交叉注意力
        self.text_to_image_attention = nn.MultiheadAttention(
            feature_dim, num_heads, dropout=dropout, batch_first=True
        )

        # 前馈网络
        self.ffn = nn.Sequential(
            nn.Linear(feature_dim * 2, feature_dim * 4),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(feature_dim * 4, feature_dim)
        )

        # 层归一化

```

```

self.layer_norm1 = nn.LayerNorm(feature_dim)
self.layer_norm2 = nn.LayerNorm(feature_dim)
self.layer_norm3 = nn.LayerNorm(feature_dim)

self.dropout = nn.Dropout(dropout)

def forward(self, visual_features, text_features):
    # 添加序列维度
    visual_features = visual_features.unsqueeze(1) # [B, 1, D]
    text_features = text_features.unsqueeze(1) # [B, 1, D]

    # 图像到文本的注意力（用文本特征查询图像特征）
    image_attended, _ = self.image_to_text_attention(
        query=text_features,
        key=visual_features,
        value=visual_features
    )
    image_attended = self.layer_norm1(text_features + self.dropout(image_attended))

    # 文本到图像的注意力（用图像特征查询文本特征）
    text_attended, _ = self.text_to_image_attention(
        query=visual_features,
        key=text_features,
        value=text_features
    )
    text_attended = self.layer_norm2(visual_features + self.dropout(text_attended))

    # 融合两个模态的特征
    combined = torch.cat([image_attended.squeeze(1), text_attended.squeeze(1)], dim=-1)
    fused = self.ffn(combined)
    fused = self.layer_norm3(fused)

    return fused

```

隐喻识别分类

将经过交叉注意力融合后的最终特征向量接入全连接分类层。分类层以多模态融合特征为输入,通过激活函数与损失函数完成模型训练与推理,最终实现图像内容的隐喻识别任务。

参考代码如下:

```

class MetaphorRecognitionModel(nn.Module):
    """完整的隐喻识别模型"""

    def __init__(self, feature_dim=768, num_classes=2):
        super().__init__()
        self.image_extractor = ImageFeatureExtractor(feature_dim=feature_dim)
        self.text_extractor = TextFeatureExtractor(feature_dim=feature_dim)

```

```

self.cross_modal_fusion = CrossModalFusion(feature_dim=feature_dim)

# 分类头
self.classifier = nn.Sequential(
    nn.Linear(feature_dim, feature_dim // 2),
    nn.ReLU(),
    nn.Dropout(0.1),
    nn.Linear(feature_dim // 2, num_classes)
)

def forward(self, pixel_values, visual_text, hidden_text):
    # 提取图像特征
    visual_features = self.image_extractor(pixel_values)

    # 合并两段文本（可以拼接或分别处理）
    # 方案 1: 拼接后统一编码
    combined_text = f"视觉描述: {visual_text}\n 深层含义: {hidden_text}"
    text_features = self.text_extractor(combined_text)

    # 方案 2: 分别编码后融合（可选）
    # visual_text_features = self.text_extractor(visual_text)
    # hidden_text_features = self.text_extractor(hidden_text)
    # text_features = (visual_text_features + hidden_text_features) / 2

    # 跨模态融合
    fused_features = self.cross_modal_fusion(visual_features, text_features)

    # 分类
    logits = self.classifier(fused_features)

    return logits

```

2.3 中英文语境下的图像情感分类任务（2-3 天完成）

2.3.1 任务描述

该任务要求根据图像本身的情感表达，结合图像的隐喻含义与表达方式，完成多维度的情感解析。核心在于通过图像所呈现的视觉信息，准确判定其传递的情感极性、强度类别。

输入：原始图像（中/英文场景）及表达方式说明。

输出：情感倾向：正面，负面，或者中性；

情感强度：评定情感表达的激烈程度等级，如低级、中级或者高级。

2.3.2 任务要求

(1)选取 2.1 中的两种不同的方法设计具体的模型，分别在中文、英文数据集上开展实验，总计完成四组对照实验，实现中英文语境下的图像情感分类任务。情感分类任务分为情感倾向与情感强度联合识别任务，选用准确率与 F1 值作为评价指标。每个数据集下情感倾向识别+情感强度识别都识别准确即认为是识别正确。

(2)对模型进行训练和验证，比较各个模型的表现，并通过调参和改进模型架构提高识别精度，将最终的结果通过图或者表格展示。

2.3.3 方法

2.5 文档撰写和答辩(1 天)

完成任务后，学生将撰写完整的实训报告，内容包括任务的解决方案、模型训练和评估过程、结果分析以及改进建议，准备答辩。